Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?
  Una lista o arreglo, ya que recorre los elementos para compararlos entre si

- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?
  Un ciclo externo q representa las pasadas y un ciclo interno que representa las comparaciones entre los elementos

- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?
  Utilizando una variable temporal o un auxiliar, que intercambie dos números de posición cuando el de la izquierda es mayor que el de la derecha.

- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?

  Cuando en una pasada completa o recorrido de la fila no se realiza ningún intercambio, lo que significa que los elementos ya están ordenados ya que los compara con cero, por lo cual queda el más grande de final

- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.

  En cada nueva pasada se compara uno menos que en la anterior, porque el elemento más grande queda en su posición definitiva al final del arreglo.

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

```
Inicio
arreglo = [34, 12, 25, 8, 19]
n = tamaño del arreglo
Para i = 0 hasta n - 2
   intercambio = falso
   Para j = 0 hasta n - i - 2
      Si arreglo[ j ] > arreglo[ j + 1] Entonces
         temp = arreglo[ j ]
         arreglo[ j ] = arreglo[ j + 1]
         arreglo[ j + 1] = temp
         intercambio = verdadero
      FinSi
   FinPara
   Si intercambio = falso Entonces
      Salir del ciclo
   FinSi
FinPara
Mostrar arreglo ordenado
Fin
```

Lluvia Estefania Villanueva Lazo.GTID244.

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
  Si, por que realiza los mismos intercambios de posición en pseudocódigo que de manera manual.

- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
  Agregar la sintaxis de Java: tipos de datos (int, boolean), llaves {}, punto y coma ; y métodos para imprimir (System.out.println).

- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
  La variable intercambio permanece en false, por lo que el algoritmo termina con el brake

- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$).

  No mucho, ya que Bubble Sort tiene una complejidad de **$O(n^2)$**, por lo que el número de comparaciones crece mucho cuando aumenta la cantidad de elementos dentro

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			

Lluvia Estefania Villanueva Lazo.GTID244.

```
package src;

// Ejercicio 1. Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código
// Autor: Lluvia Estefania Villanueva Lazo
// Fecha: Jueves 05 de Noviembre de 2026
// GTID244

public class BubbleSort {

    public static void main(String[] args) {

// Caso 1: Arreglo original
        int[] arreglo1 = {34, 12, 25, 8, 19};

// Caso 2: Arreglo ya ordenado
        int[] arreglo2 = {8, 12, 19, 25, 34};

// Caso 3: Arreglo en orden inverso
        int[] arreglo3 = {34, 25, 19, 12, 8};

        System.out.println("=== ARREGLO ORIGINAL ===");
        bubbleSort(arreglo1);

        System.out.println("\n=== ARREGLO YA ORDENADO ===");
        bubbleSort(arreglo2);

        System.out.println("\n=== ARREGLO EN ORDEN INVERSO ===");
        bubbleSort(arreglo3);
    }

    public static void bubbleSort(int[] numeros) {

// Obtiene la cantidad de elementos del arreglo
        int n = numeros.length;

// Variable para saber si hubo intercambios en una pasada
        boolean intercambio;

// Contadores
        int comparaciones = 0;
        int intercambios = 0;
        int pasadas = 0;

// Muestra el arreglo original
        System.out.print("Arreglo inicial: ");
        for (int num : numeros) {
            System.out.print(num + " ");
        }
        System.out.println("\n");

// Ciclo externo: controla las pasadas
        for (int i = 0; i < n - 1; i++) {

// Al inicio de cada pasada se supone que no habrá intercambios
            intercambio = false;
```

Lluvia Estefania Villanueva Lazo.GTID244.

```
// Incrementa el contador de pasadas
            pasadas++;

// Ciclo interno: realiza las comparaciones
            for (int j = 0; j < n - i - 1; j++) {

// Cuenta cada comparación realizada
                comparaciones++;

// Si el número actual es mayor que el siguiente
                if (numeros[j] > numeros[j + 1]) {

// Intercambio usando una variable temporal
                    int temp = numeros[j];
                    numeros[j] = numeros[j + 1];
                    numeros[j + 1] = temp;

// Cuenta el intercambio realizado
                    intercambios++;

// Indica que sí hubo intercambio en esta pasada
                    intercambio = true;
                }
            }

// Muestra el arreglo después de cada pasada
            System.out.print("Pasada " + pasadas + ": ");

            for (int num : numeros) {
                System.out.print(num + " ");
            }

            System.out.println();

// Si no hubo intercambios, el arreglo ya está ordenado
            if (!intercambio) {
                break;
            }
        }

 // Muestra el arreglo ordenado
        System.out.print("\nArreglo ordenado: ");
        for (int num : numeros) {
            System.out.print(num + " ");
        }

// Muestra todo
        System.out.println("\n");
        System.out.println("Total de comparaciones: " + comparaciones);
        System.out.println("Total de intercambios: " + intercambios);
        System.out.println("Total de pasadas: " + pasadas);
    }
}
```

Lluvia Estefania Villanueva Lazo.GTID244.

```
C:\U2ACT1\src>javac BubbleSort.java

C:\U2ACT1\src>java BubbleSort.java
=== ARREGLO ORIGINAL ===
Arreglo inicial: 34 12 25 8 19

Pasada 1: 12 25 8 19 34
Pasada 2: 12 8 19 25 34
Pasada 3: 8 12 19 25 34
Pasada 4: 8 12 19 25 34

Arreglo ordenado: 8 12 19 25 34

Total de comparaciones: 10
Total de intercambios: 7
Total de pasadas: 4

=== ARREGLO YA ORDENADO ===
Arreglo inicial: 8 12 19 25 34

Pasada 1: 8 12 19 25 34

Arreglo ordenado: 8 12 19 25 34

Total de comparaciones: 4
Total de intercambios: 0
Total de pasadas: 1

=== ARREGLO EN ORDEN INVERSO ===
Arreglo inicial: 34 25 19 12 8

Pasada 1: 25 19 12 8 34
Pasada 2: 19 12 8 25 34
Pasada 3: 12 8 19 25 34
Pasada 4: 8 12 19 25 34

Arreglo ordenado: 8 12 19 25 34

Total de comparaciones: 10
Total de intercambios: 10
Total de pasadas: 4
```